

Sovereign AI Workbench – Implementation Guide

Sep 24, 2026 · @CHARUSAT

Overview

We are building an offline AI assistant for MRPL (Problem Statement 26117). It runs fully on our own laptop, with no GPU and no internet, and two developers build it in parallel.

What the system does:

- Picks the right AI model for each task (coding vs documents vs images).
- Works like an agent: makes a plan, uses tools, checks results, repeats.
- Reads scanned PDFs, photos and drawings (OCR + vision model).
- Makes real files: Word approval notes, Excel sheets, PPT, tested code.
- Searches company SOPs and manuals (knowledge base).
- Proves that no data leaves the machine (firewall + live network monitor).

The 3 demo scenarios we must show:

#	Scenario	Input	Output
A	Inspection report to approval note	Scanned PDF	Word file (.docx)
B	Engineering calculation code	Text request	Python code + passing tests in sandbox
C	P&ID drawing to tag list	Image	Excel file (.xlsx)

Team split in one line each:

- **Dev A (Track A)** makes it work: backend, models, tools, agent, network proof.
- **Dev B (Track B)** makes it visible: UI, mock server, demo data, PPT, demo scripts.

```
flowchart LR
```

```

  U[User browser] --> UI[Streamlit UI<br/>Dev B]
  UI -->|HTTP 127.0.0.1| API[FastAPI backend<br/>Dev A]
  API --> R[Router]
  R --> AG[Agent loop]
  AG --> T[Tools: OCR, RAG,<br/>sandbox, Office]
  AG --> OL[Ollama models]
  T --> SB[Dockersandbox<br/>no network]
  T --> DB[ChromaDB]
```

The UI never talks to models directly. It only calls the backend API, which is defined in `shared/contracts.py`.

How to use this doc: Follow the steps in order. Step 1 and Step 2 are done by both devs on Day 0. Then each dev follows only their own ticket list. Every step says what to run and what result to expect.

Words used in this doc:

Word	Simple meaning
LLM / model	The AI brain that reads and writes text
Ollama	Program that runs models on our laptop at <code>127.0.0.1:11434</code>
Agent	AI that plans, uses tools, checks, and repeats until done
Tool	A Python function the AI is allowed to call
OCR	Reading text from a picture (we use Tesseract)
RAG / knowledge base	Search our own documents, then give the best parts to the AI
Embedding	Numbers that represent the meaning of text, used for search
Sandbox	A locked Docker box where AI-written code runs safely
Mock server	A fake backend with ready answers, so the UI can be built early

Word	Simple meaning
Contract	The fixed agreement on API shapes between Dev A and Dev B
Lock file	A file listing exact package versions so both laptops match

Step 1: Downloads and installs (both devs, Day 0)

Both laptops must have every tool and model below before coding starts. Our target machine is Windows 11, 16 GB RAM, no GPU, 30 GB free disk.

Open **PowerShell as Administrator** for all commands (Start, type PowerShell, right-click, Run as administrator).

1.1 Check the PC

1. Press `Ctrl + Shift + Esc`, go to Performance, then CPU. Note the number of cores.
2. Check **Virtualization: Enabled**. Docker needs it.
3. If it says Disabled: restart, open BIOS (F2, F10 or Del), turn on Intel VT-x or AMD SVM.
4. Make sure C: has at least 30 GB free.

1.2 Install the tools

```
winget install -e --id Git.Git
winget install -e --id Python.Python.3.11
winget install -e --id Ollama.Ollama
winget install -e --id Docker.DockerDesktop
winget install -e --id UB-Mannheim.TesseractOCR
```

Tool	Why we need it
Git	Code versions and sharing between the two devs
Python 3.11	Our language; all our libraries support 3.11 well
Ollama	Runs the AI models locally
Docker Desktop	Runs the code sandbox

Tool	Why we need it
Tesseract	OCR engine for scanned pages

Close PowerShell and open a new admin window. Then check:

```
git --version
python --version
ollama --version
docker --version
```

Expected: each prints a version, for example `Python 3.11.x`.

If `python` opens the Microsoft Store: Settings, Apps, Advanced app settings, App execution aliases, turn off both python aliases. Open a new window.

1.3 Add Tesseract to PATH

```
[Environment]::SetEnvironmentVariable("Path", $env:Path + ";C:\Program
Files\Tesseract-OCR", "User")
```

Open a new window and run `tesseract --version`. **Expected:** `tesseract v5.x`. If not found, check the folder `C:\Program Files\Tesseract-OCR` exists.

1.4 WSL2 and Docker memory limit

WSL2 is a small Linux inside Windows that Docker needs. By default it can take 8 GB of RAM, which leaves no room for the models, so we limit it.

```
wsl --install
```

Restart the PC. Then create the config file:

```
notepad "$env:USERPROFILE\.wslconfig"
```

Click Yes to create it, paste this, and save:

```
[wsl2]
memory=4GB
processors=4
swap=2GB
```

Apply it with `wsl --shutdown`, then open Docker Desktop and wait for **Engine running**.

If Notepad saves `.wslconfig.txt`: in Save As choose "All files" and type the name in quotes: `".wslconfig"`.

1.5 Ollama settings for CPU

```
setx OLLAMA_MAX_LOADED_MODELS 2
setx OLLAMA_NUM_PARALLEL 1
setx OLLAMA_KEEP_ALIVE 30m
```

Setting	Meaning
MAX_LOADED_MODELS 2	Max 2 models in RAM at once
NUM_PARALLEL 1	One request at a time, saves RAM
KEEP_ALIVE 30m	Keep a model loaded 30 minutes, no slow reload

Restart Ollama so it reads them: right-click the Ollama tray icon, Quit, then open Ollama again.

1.6 Download the models (about 7 GB)

```
ollama pull qwen3.5:4b
ollama pull qwen2.5-coder:3b
ollama pull bge-m3
ollama list
```

Expected: all 3 models appear in `ollama list`.

1.7 Speed test

```
ollama run qwen3.5:4b --verbose --think=false "Explain OCR in 3 simple lines."
ollama ps
```

- Find `eval rate: X tokens/s`. 8 or more is great, 4 to 8 is OK, below 4 means we switch to `qwen3.5:2b`.
- `ollama ps` should show `100% CPU`. That is correct for our laptops.
- Write the number in the team chat.

If **--think** is unknown: Ollama is old, run `winget upgrade Ollama.Ollama`.

1.8 Build the code sandbox image

The repo does not exist yet, so we build the image from a temporary folder. The same Dockerfile is committed to `sandbox/Dockerfile` in Step 2. Run `mkdir`

`C:\projects\sandbox-build`, then `notepad C:\projects\sandbox-build\Dockerfile` (no extension) and paste:

```
FROM python:3.11-slim
RUN pip install --no-cache-dir numpy pandas sympy pytest
RUN useradd -m runner
USER runner
WORKDIR /work
```

- `FROM`: small official Python image.
- `RUN pip install`: libraries the AI code may need; `pytest` runs tests.
- `useradd` + `USER runner`: code runs as a normal user, not admin.
- `WORKDIR /work`: folder where code files go.

Build and test (Docker Desktop must be running):

```
docker build -t wb-sandbox:1.0 C:\projects\sandbox-build
docker run --rm --network none wb-sandbox:1.0 python -c "print(2+2)"
docker run --rm --network none wb-sandbox:1.0 python -c "import
urllib.request; urllib.request.urlopen('https://google.com')"
```

Expected: the first run prints `4`. The second run shows an **error**. That error is good: it proves AI code cannot reach the internet.

Save a backup of the image:

```
mkdir C:\projects\offline_kit
docker save -o C:\projects\offline_kit\wb-sandbox.tar wb-sandbox:1.0
```

1.9 Offline test

Turn Wi-Fi off and run:

```
ollama run qwen2.5-coder:3b "Write a Python function that adds two numbers."
docker run --rm --network none wb-sandbox:1.0 python -c "print('sandbox ok')"
tesseract --version
```

Expected: all three work with no internet. Step 1 is done.

Step 1 checklist

- ☐ All tools show versions
- ☐ Tesseract on PATH
- ☐ WSL limited to 4 GB, Docker engine running
- ☐ Ollama settings set and Ollama restarted
- ☐ 3 models downloaded
- ☐ Speed test number shared
- ☐ Sandbox image built, internet test fails as expected
- ☐ Offline test passed

Step 2: Repo setup on Day 0

Dev A creates the repo and the lock file once. Dev B installs only from that lock file, so both laptops have exactly the same package versions.

Why a lock file: if Dev A has one version of a library and Dev B has another, code can work on one laptop and break on the other. The lock file records exact versions. It is the only truth.

Rule: do not create the project inside OneDrive or Desktop. OneDrive syncs thousands of library files and breaks things.

2.1 Dev A: create the repo

```
mkdir C:\projects\sovereign-workbench
cd C:\projects\sovereign-workbench
git init
python -m venv .venv
.\.venv\Scripts\Activate.ps1
```

Expected: the prompt starts with `(.venv)`.

If you see "running scripts is disabled": run `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`, then activate again.

2.2 Dev A: create all folders

```
$dirs =
"backend\tools","backend\flows","ui\components","ui\pages","mock","monitor","
sandbox","templates","scripts","config","demo\inputs","docs\ppt","data\kb","d
ata\chroma","data\cache","workspace","logs","tests\contract","tests\track_a",
"tests\track_b","shared\fixtures\files"
$dirs | ForEach-Object { New-Item -ItemType Directory -Force -Path $_ | Out-
Null }
New-Item -ItemType File -Force backend\__init__.py,
backend\tools\__init__.py, backend\flows\__init__.py, ui\__init__.py,
ui\components\__init__.py, mock\__init__.py, monitor\__init__.py,
shared\__init__.py, tests\__init__.py | Out-Null
copy C:\projects\sandbox-build\Dockerfile sandbox\Dockerfile
```

The `__init__.py` files make each folder a Python package, so imports like `from shared.contracts import TaskState` work.

2.3 Final folder structure

sovereign-workbench/	
AGENTS.md	rules for AI coding agents (both)
CLAUDE.md	one line: follow AGENTS.md
.env.example	all settings (both)
requirements.in	top-level packages
requirements.lock.txt	exact versions (install only from this)
VERSIONS.md	tool and model versions
config/models.yaml	model registry + router rules (Dev A)
shared/contracts.py	API contract (both)
shared/fixtures/	mock data (both)
backend/	FastAPI, router, agent, tools (Dev A)
backend/tools/	files, documents, knowledge, sandbox, office
backend/flows/	code flow, guided flows
monitor/	network monitor (Dev A)
sandbox/Dockerfile	sandbox image (Dev A)
templates/	Word template (Dev A)
ui/	Streamlit app (Dev B)
mock/	mock server (Dev B)
demo/	demo inputs + talk track (Dev B)

docs/ppt/

scripts/

tests/contract/

tests/track_a/, tests/track_b/

data/kb/

workspace/, logs/

presentation (Dev B)

doctor, ingest, firewall, start_demo, e2e

contract tests (both)

SOP PDFs; data/chroma, data/cache (ignored)

runtime only (ignored)

2.4 Dev A: add the starter files

Copy the 6 starter-kit files (shared in chat) into these exact paths:

File	Path in repo
contracts.py	shared/contracts.py
inspection_note.json	shared/fixtures/inspection_note.json
.env.example	.env.example
models.yaml	config/models.yaml
AGENTS.md	AGENTS.md
requirements.in	requirements.in

Create `CLAUDE.md` with one line: Read and follow AGENTS.md strictly.

Create `.gitignore`:

.venv/

.env

__pycache__/

workspace/

logs/

data/chroma/

data/cache/

offline_kit/

*.tar

2.5 Dev A: create the lock file and versions file

```
pip install -r requirements.in
pip freeze > requirements.lock.txt
python --version > VERSIONS.md
ollama --version >> VERSIONS.md
docker --version >> VERSIONS.md
tesseract --version >> VERSIONS.md
ollama list >> VERSIONS.md
copy .env.example .env
```

`VERSIONS.md` records exact tool versions and model IDs (the long code in `ollama list`).

Save all packages offline, in case you must reinstall without internet:

```
mkdir C:\projects\offline_kit\wheels
pip download -r requirements.lock.txt -d C:\projects\offline_kit\wheels
```

2.6 Dev A: first commit and push

Create a **private** GitHub repo first, then:

```
git add .
git commit -m "Day0: contracts, env, lock, skeleton"
git branch -M main
git remote add origin <your-private-repo-url>
git push -u origin main
```

2.7 Dev B: clone and install from the lock

```
cd C:\projects
git clone <your-private-repo-url> sovereign-workbench
cd sovereign-workbench
python -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r requirements.lock.txt
copy .env.example .env
```

Now compare with `VERSIONS.md` :

- Ollama version different: `winget install -e --id Ollama.Ollama --version <version from VERSIONS.md>`.

- A model ID different in `ollama list`: both devs run `ollama pull <model>` again and compare.

2.8 Both: create your branch

```
git checkout -b track-a    # Dev A
git checkout -b track-b    # Dev B
git push -u origin track-a  # or track-b
```

2.9 The .env file

`.env.example` holds every setting: ports, paths, timeouts, limits, telemetry switches. Each dev copies it to `.env` (never committed).

- Code never hard-codes a port, path or model name. Everything comes from `.env`.
- The only line that differs at first is `WB_API_URL`. Dev B uses `http://127.0.0.1:8001` (mock). After integration both use `http://127.0.0.1:8000` (real).
- `WB_API_HOST=127.0.0.1` means the server listens only on this laptop. Never use `0.0.0.0`.
- The last block (`ANONYMIZED_TELEMETRY=False`, `HF_HUB_OFFLINE=1` and others) stops libraries from sending data home. The sovereign proof depends on it.
- A new setting is added to `.env.example` only through a shared PR.

Step 2 checklist

- ☐ Repo on GitHub with the Day-0 commit
- ☐ Both laptops: venv active, installed from `requirements.lock.txt`
- ☐ `VERSIONS.md` matches on both laptops
- ☐ Both have `.env`
- ☐ Branches `track-a` and `track-b` pushed

Step 3: Team rules (read before writing any code)

Nobody edits the other dev's folders, and nobody edits shared files alone. If we follow this, merge conflicts cannot happen.

3.1 Folder ownership

Folder or file	Owner
backend/, sandbox/, config/, monitor/, templates/	Dev A
scripts/doctor.py, scripts/ingest.py, scripts/firewall_*.ps1	Dev A
ui/, mock/, demo/, docs/ppt/	Dev B
scripts/start_demo.ps1, scripts/e2e_run.py	Dev B
shared/, tests/contract/, AGENTS.md, .env.example, requirements.lock.txt	Both (PR, both approve)
tests/track_a/	Dev A
tests/track_b/	Dev B

3.2 Git flow

1. `main` always works. Nobody pushes to `main` directly.
2. Dev A works on `track-a`, Dev B on `track-b`.
3. Every morning: `git pull origin main`, then `git merge main` into your branch.
4. Once a day at a fixed time (for example 9 PM): open a PR from your branch to `main`.
The other dev looks at it and merges.
5. Commit small and often, with clear messages like `A4: OCR fallback to vision`.

```

flowchart LR
    A[track-a] -->|daily PR| M[main]
    B[track-b] -->|daily PR| M
    M -->|every morning merge| A
    M -->|every morning merge| B

```

3.3 Rules for AI coding agents (Claude Code, Antigravity)

AI agents like to "helpfully fix" other files. `AGENTS.md` stops that. Its key rules:

1. Never edit `shared/contracts.py`, fixtures, `.env.example` or the lock file.
2. Only edit files in your own track's folders.

3. No network calls except to 127.0.0.1. No telemetry, no downloads at runtime.
4. No new packages. Requests go in `docs/dependency_requests.md`.
5. Import all API types from `shared.contracts`. Never redefine them.
6. Read settings only from `.env` through the settings file.
7. Every new function gets a pytest test. Run `pytest -q` before finishing.
8. Use `pathlib.Path` for Windows paths.

Start every agent session with this prompt (change the track and ticket):

```
Read AGENTS.md and shared/contracts.py first.
You are Track A. Do ticket A3 from the implementation guide.
Only edit files in Track A folders. Show me the test results at the end.
```

3.4 Changing the contract or packages

1. Write the request in `docs/contract_change_requests.md` or `docs/dependency_requests.md`.
2. Both agree in chat.
3. One PR changes the file. Bump `CONTRACT_VERSION` in `contracts.py`.
4. For a new package: add it to `requirements.in`, Dev A regenerates the lock, both run `pip install -r requirements.lock.txt`.
5. After Day 0, only new optional fields are allowed in the contract. Never rename or delete.

3.5 Daily routine

- ☐ Morning: pull `main`, merge into your branch
- ☐ Morning: `python scripts/doctor.py` shows all PASS (after ticket A0 exists)
- ☐ During the day: one ticket at a time, tests pass before moving on
- ☐ Evening: PR to `main`, short update in team chat (done, next, blocked)

Step 4: The shared contract

`shared/contracts.py` is the boss: it defines every endpoint and every request and response shape. Dev B builds against a mock that follows it, Dev A builds the real backend that follows it, and on integration day we only change one URL.

4.1 Endpoints

All endpoints are under `/api` on `127.0.0.1`. Real backend on port 8000, mock on port 8001.

Method	Path	Request	Response	Used for
GET	<code>/api/health</code>	none	<code>HealthResponse</code>	Sidebar status lights
GET	<code>/api/models</code>	none	<code>list[ModelInfo]</code>	Which models are loaded
POST	<code>/api/files</code>	multipart, field <code>file</code>	<code>FileRef</code>	Upload a file
POST	<code>/api/route</code>	<code>RouteRequest</code>	<code>RouteDecision</code>	Router badge preview
POST	<code>/api/tasks</code>	<code>TaskCreate</code>	<code>TaskCreated (202)</code>	Start a task
GET	<code>/api/tasks/{id}</code>	none	<code>TaskState</code>	Final answer and files
GET	<code>/api/tasks/{id}/events?after=N</code>	none	<code>EventsPage</code>	Live agent steps
POST	<code>/api/tasks/{id}/cancel</code>	none	<code>TaskState</code>	Stop button
GET	<code>/api/artifacts/{id}</code>	none	file bytes	Download Word/Excel/PPT
GET	<code>/api/network/status</code>	none	<code>NetworkStatus</code>	Network monitor panel
POST	<code>/api/network/probe</code>	<code>ProbeRequest</code>	<code>ProbeResult</code>	Try-to-reach-Google button
GET	<code>/api/kb/stats</code>	none	<code>KBStats</code>	Knowledge base info
POST	<code>/api/kb/search</code>	<code>KBSearchRequest</code>	<code>KBSearchResponse</code>	Search debugging
GET	<code>/api/audit?task_id=&limit=</code>	none	<code>list[AuditRecord]</code>	Audit log page
POST	<code>/api/admin/prewarm</code>	none	<code>PrewarmResult</code>	Load models before demo

Every error response has the body `ApiError`, with a `code` from `ERROR_CODES` in `contracts.py`.

4.2 How a task runs: polling

The UI asks every 1 second, "any new events after number N?" We use polling, not WebSockets, because it is simple and does not break silently inside Streamlit. A 1-second delay is invisible when each step takes 5 to 30 seconds on CPU.

```
sequenceDiagram
    participant UI
    participant API
    UI->>API: POST /api/files
    API-->>UI: FileRef (file_id)
    UI->>API: POST /api/tasks
    API-->>UI: TaskCreated (task_id)
    loop every 1 second
        UI->>API: GET events?after=next_seq
        API-->>UI: EventsPage (events, next_seq, done)
    end
    UI->>API: GET /api/tasks/{id}
    API-->>UI: TaskState (answer, artifacts)
```

- Each event has a number `seq` starting at 1. `after=0` returns all events.
- The UI keeps the last `next_seq` and sends it as `after` next time.
- When `done` is true, the UI stops polling and fetches the final `TaskState` once.
- The backend runs **one task at a time**. A second task gets status `queued`. Two tasks at once on a 16 GB CPU laptop would be slow and could run out of RAM.

4.3 Event types

Every agent action becomes an `AgentEvent`. Dev B draws them as a timeline. The `data` keys are fixed:

Type	data keys	UI shows
<code>route</code>	<code>decision</code>	Router badge
<code>plan</code>	<code>steps</code>	Plan checklist
<code>step_start</code>	<code>index</code> , <code>title</code>	New step in timeline
<code>llm_call</code>	<code>model_id</code> , <code>purpose</code> , <code>duration_ms</code> , <code>tokens_out</code>	Model used + time

Type	data keys	UI shows
<code>tool_call</code>	<code>tool</code> , <code>args</code>	Tool being called
<code>tool_result</code>	<code>tool</code> , <code>ok</code> , <code>summary</code> , <code>duration_ms</code>	Result line, green or red
<code>artifact</code>	<code>artifact</code>	Download button
<code>log</code>	<code>level</code> , <code>text</code>	Small grey note
<code>final</code>	<code>answer</code>	Final answer box
<code>error</code>	<code>error</code>	Red error box

Normal order: route, plan, then for each step: `step_start`, `tool_call` or `llm_call`, `tool_result`. Then artifact and final (or error).

4.4 Task statuses

```
stateDiagram-v2
    [*] --> queued
    queued --> running
    running --> succeeded
    running --> failed
    running --> cancelled
    queued --> cancelled
```

4.5 Deliverable schemas (what the AI fills)

The AI never builds Office files directly. It fills a JSON form, and our code builds the file from a template. Ollama can force the reply to match a JSON schema, so the form is always valid.

Schema	Used in	Key fields
ApprovalNote	Scenario A	subject, background, findings, sop_references, recommendation
Finding	Scenario A	item, observation, severity, source_page
CodeResult	Scenario B	code, tests, passed, failed, attempts
PidTagList / PidTag	Scenario C	tag, equipment_type, description, tile

`ref_no` and `date` in `ApprovalNote` are filled by code, never by the AI.

4.6 The mock server

- Fixture files in `shared/fixtures/` hold a fake but realistic full run of each scenario.
- `inspection_note.json` is already written. Dev B writes `code_calc.json` and `pid_tags.json` in the same format (ticket B0).
- The mock releases one event every `release_every_s` seconds (1.5), so the UI feels like a real run.
- The mock replaces placeholders: `$route`, `$plan`, `$artifact` with those objects from the same file, `$task_id` with the real task id, `$now` with the current time.
- Download of the mock artifact returns the file at `artifact_file`.

4.7 Contract tests

One test folder, `tests/contract/`, calls every endpoint and checks every response with the Pydantic models from `contracts.py`. We run the same tests against the mock and the real backend:

```
pytest tests/contract --base-url http://127.0.0.1:8001 # mock
pytest tests/contract --base-url http://127.0.0.1:8000 # real
```

If both pass, the UI cannot break when we switch from mock to real. If the real backend fails a test, it is a Dev A bug: the contract decides, not opinions.

4.8 Model registry (`config/models.yaml`)

The router reads this file. It has 3 layers:

1. **Rules** (checked first): image attached, scanned PDF, coding keywords, document keywords.
2. **Similarity**: compare the request's embedding with example sentences per task type. Match above 0.55 wins.
3. **Default**: general model.

To add or upgrade a model, change only this file. Each model has a comment with its production replacement.

Step 5: Dev A tickets (backend, models, pipeline)

Dev A does 11 tickets in order, A0 to A10. A3 (the API skeleton) must come early so Dev B can test against the real backend soon. Each ticket is one AI agent session, and its tests must pass before the next ticket starts.

A0: Settings and doctor script

- **Build**: `backend/settings.py` using `pydantic-settings`, which loads `.env` into one typed `Settings` object. `scripts/doctor.py`, a health check.
- **Doctor checks**: Python is 3.11; `pip freeze` matches `requirements.lock.txt`; Ollama answers at `OLLAMA_HOST`; the 3 models exist; Tesseract path works; Docker is running; image `wb-sandbox:1.0` exists; `.env` has every key from `.env.example`.
- **Done when**: `python scripts/doctor.py` prints PASS for every line on both laptops.

A1: LLM client

- **Build**: `backend/llm_client.py` with these functions:
 - `chat(model_id, messages, tools=None, images=None)` returns text and tool calls.
 - `chat_json(model_id, messages, schema)` sends the Pydantic schema as Ollama format, validates the reply, and retries once with the error message.
 - `embed(texts)` uses the `embed` model.
- **Every call**: `think=false`, `num_ctx` and `temperature` from settings, a timeout, one retry, and an audit record.
- **Fallback parser**: small models sometimes print a tool call as text (for example `<tool_call>...`). Detect that text and turn it into a real tool call.

- **Done when:** `tests/track_a/test_llm_client.py` passes: a text answer, a valid `ApprovalNote` JSON answer, an image answer, an embedding with length above 0.

A2: Registry and router

- **Build:** `backend/registry.py` loads `config/models.yaml`. `backend/router.py` returns a `RouteDecision` using rules, then similarity, then default.
- **Similarity:** embed the example sentences once at startup and keep them in memory. Use cosine similarity. Above `similarity_threshold` wins.
- **Done when:** a test with 20 prompts (5 each: coding, document, vision, general) gets at least 18 right, and each decision has a clear `reason`.

A3: API skeleton (early)

- **Build:** `backend/main.py` with every endpoint from Step 4. `backend/task_store.py` holds tasks and their event lists in memory, protected by a thread lock. One worker thread takes tasks from a queue.
- **For now** the agent is an echo: it sends route, one fake step, and final.
- Add header `X-Contract-Version` on every response. Bind to `127.0.0.1:8000`.
- **Run:** `uvicorn backend.main:app --host 127.0.0.1 --port 8000`
- **Done when:** `pytest tests/contract --base-url http://127.0.0.1:8000` passes. Then tell Dev B: "early integration ready".

A4: Files and documents

- **Build:** `backend/tools/files.py`: save uploads to `workspace/<session>/`, and reject any path outside `workspace/`. Check size and file type.
- **Build:** `backend/tools/documents.py`, with this order:
 1. PDF has a text layer: take the text directly.
 2. No text layer: render pages at `WB_OCR_DPI` (200) with PyMuPDF, then Tesseract OCR.
 3. Handwriting, tables, or poor OCR: send the page image to the vision model.
 4. P&ID images: cut into 4 overlapping tiles, send each tile.
- Resize every image to max `WB_VISION_MAX_PX` (1024) before sending. Cache results by file hash in `data/cache/`.
- **Done when:** 3 sample scanned PDFs give readable text, and a path like `..\..\Windows` is rejected.

A5: Sandbox and code flow

- **Build:** `backend/tools/sandbox.py` using the Docker Python SDK: `network_disabled=True`, `mem_limit` from `WB_SANDBOX_MEM`, one CPU, and kill the container after `WB_SANDBOX_TIMEOUT_S`.
- **Build:** `backend/flows/code_flow.py`: coder model writes code and tests, the sandbox runs pytest, a failure sends the error back, maximum 3 attempts. Returns `CodeResult`.
- **Demo task:** a function for pipe wall thickness from pressure, diameter and allowable stress, with tests and the calculation steps printed.
- **Done when:** the demo task passes, a buggy seed code gets fixed by retry, and an internet call inside the sandbox fails.

A6: Knowledge base

- **Build:** `scripts/ingest.py` reads every PDF in `data/kb/` (using A4 for scanned ones), cuts it into chunks of about 800 tokens with 100 overlap, embeds them with `bge-m3`, and stores them in Chroma at `WB_CHROMA_DIR` with file name and page.
- **Build:** `backend/tools/knowledge.py` with `search(query, top_k)` returning `KBHit` items.
- **Two traps:** always pass our own embeddings, because Chroma's default embedding function downloads a model from the internet; and create the client with `anonymized_telemetry=False`.
- **Done when:** 10 known questions find the right chunk in the top 4 at least 8 times.

A7: Deliverables

- **Build:** `templates/approval_note.docx` with fixed sections: Ref no, Date, Subject, Background, Findings table, SOP references, Recommendation, Cost implication, Approval signature block.
- **Build:** `backend/tools/office.py` with `make_word(ApprovalNote)`, `make_excel(PidTagList or findings)`, `make_ppt(summary)`. Code fills `ref_no` (like `INSP-2026-001`) and `date`.
- **Done when:** generated files open in Word and Excel with no repair warning.

A8: Agent loop and guided flows

- **Build:** `backend/agent.py`:
 1. Route the request, send a `route` event.

2. Ask for a plan as JSON, send a `plan` event.
 3. Loop: model picks a tool, we run it, send events, give the result back.
 4. Stop at a final answer, at `WB_AGENT_MAX_STEPS` (8), at `WB_AGENT_TIMEOUT_S` (600), or on cancel.
- Every tool is wrapped in try/except. An error becomes an `error` event, never a crashed server.
 - **Build:** `backend/flows/guided.py` : fixed pipelines for the 3 scenarios, using the same tools in a fixed order. Used when `mode` is `guided`.
 - **Done when:** all 3 scenarios succeed through the API in both `agent` and `guided` mode.

A9: Network proof and audit

- **Build:** `monitor/net_monitor.py` : a background thread that reads `psutil.net_connections()` every `WB_NET_POLL_S` second. Ignore listening sockets and loopback addresses. Count every other remote address as external and log it.
- **Build:** `/api/network/status`, `/api/network/probe` (tries to open the target URL with a 3 second timeout), and `backend/audit.py` writing `logs/audit.jsonl`.
- **Build firewall scripts:**

```
# scripts/firewall_block.ps1 (run as admin)
Set-NetFirewallProfile -Profile Domain,Public,Private -DefaultOutboundAction
Block
# scripts/firewall_restore.ps1 (run as admin)
Set-NetFirewallProfile -Profile Domain,Public,Private -DefaultOutboundAction
Allow
```

- Block stops every outgoing connection from the PC. Traffic to 127.0.0.1 still works, so our app runs. Always restore after the demo.
- **Done when:** with the firewall on, the probe returns `reachable: false`, and `external_seen_since_start` stays 0 through all 3 scenarios.

A10: Performance

- **Build:** `/api/admin/prewarm` loads `general` and `embed` models. Write a timing table per scenario in `docs/perf.md`.
- **Tuning options:** lower `WB_NUM_CTX`, shorter prompts, fewer agent steps, smaller demo files.
- **Done when:** each scenario finishes in under 5 minutes on our laptop.

Dev A checklist

- ☐ A0 Settings + doctor
- ☐ A1 LLM client
- ☐ A2 Router
- ☐ A3 API skeleton (tell Dev B)
- ☐ A4 Documents + OCR
- ☐ A5 Sandbox + code flow
- ☐ A6 Knowledge base
- ☐ A7 Office files
- ☐ A8 Agent + guided flows
- ☐ A9 Network proof + audit
- ☐ A10 Performance

Step 6: Dev B tickets (UI, client, demo)

Dev B does 10 tickets in order, B0 to B9, and never waits for Dev A because everything starts on the mock server. B6 (demo data) must reach Dev A by the end of Day 1.

B0: Mock server

- **Build:** `mock/mock_server.py` : a FastAPI app on port 8001 with every endpoint from Step 4, returning data that matches `contracts.py`.
- **Tasks:** pick the fixture from the `scenario` field (or from the message text), release one event every `release_every_s` seconds, replace the `$` placeholders.
- **Write fixtures:** `shared/fixtures/code_calc.json` and `shared/fixtures/pid_tags.json`, same format as `inspection_note.json`. Put small sample output files in `shared/fixtures/files/` (a sample docx, xlsx, py).
- **Run:** `uvicorn mock.mock_server:app --host 127.0.0.1 --port 8001`
- **Done when:** B1 contract tests pass against the mock.

B1: Contract tests (shared)

- **Build:** `tests/contract/conftest.py` with a `--base-url` pytest option. `tests/contract/test_endpoints.py` calls every endpoint and validates each response with the matching Pydantic model.

- Include one full flow: upload a file, create a task, poll events until `done`, get the task, download the artifact.
- **Done when:** it passes against the mock. Open a PR; both devs approve because it is a shared file.

B2: API client

- **Build:** `ui/config.py` (reads `WB_API_URL` from `.env`) and `ui/api_client.py`: one typed function per endpoint using `httpx`.
- Timeout 10 seconds (uploads 60 seconds). Errors turn into an `ErrorInfo` object. The client never throws an exception into the UI.
- **Done when:** `tests/track_b/test_api_client.py` passes against the mock.

B3: UI shell

- **Build:** `ui/app.py` in Streamlit, wide layout:
 - **Sidebar:** health lights (Ollama, sandbox, Tesseract, knowledge base), contract version check, mode toggle (Agent or Guided), 3 scenario buttons, Reset session.
 - **Main left:** chat and file upload.
 - **Main right:** router badge, plan, timeline, files, network panel.
- Create `.streamlit/config.toml` with `[browser]` then `gatherUsageStats = false`.
- **Run:** `streamlit run ui/app.py --server.address 127.0.0.1 --server.port 8501`
- **Done when:** the UI opens and all health lights are green on the mock.

B4: Task run and live timeline

- **Build:** `ui/components/timeline.py`: upload, create task, then poll with `st.fragment(run_every=1)` so only that panel refreshes each second.
- One icon and one line per event type (see Step 4.3). Show the final answer when done. Add a Cancel button.
- **Done when:** a full mock scenario plays live, with no page freeze.

B5: Router badge, plan view, artifacts

- **Build:** `ui/components/router_badge.py` (model name, reason, layer), `ui/components/plan_view.py` (step checklist that ticks as steps finish), `ui/components/artifacts.py` (download buttons and previews).

- **Previews:** docx shows the `preview` text; xlsx is shown as a table using pandas; py shows code.
- **Done when:** all 3 mock scenarios show the badge, the plan and the downloads.

B6: Demo data pack

- **Collect public files only** (no real MRPL data):
 - `demo/inputs/` : 2 scanned inspection reports (1 to 2 pages each), 2 P&ID images.
 - `data/kb/` : 5 to 10 public SOP or safety manual PDFs.
- Write `demo/expected.md` : what a correct output must contain for each input (findings, tags, test count).
- **Done when:** files are pushed and Dev A is told, by the end of Day 1.

B7: Network and audit pages

- **Build:** `ui/components/network_panel.py` : big number "External connections: 0" in green (red if above 0), a table of connections, and a "Try to reach Google" button that calls the probe.
- **Build:** `ui/pages/audit.py` : audit records table, filter by task.
- **Done when:** it works on the mock, and a red state shows when the mock returns `external_count` above 0.

B8: Resilience

- Backend-down banner with auto-retry every 3 seconds.
- Contract version mismatch banner (compare `/api/health` with `CONTRACT_VERSION`).
- Every API error shown as a friendly message, never a Python traceback.
- Reset session button clears the UI state.
- **Done when:** killing the mock during a task shows the banner with no crash, and the UI recovers after restart.

B9: Demo pack

- `scripts/start_demo.ps1` : one-click start (see Step 8).
- `scripts/e2e_run.py` : runs all 3 scenarios through the API N times and checks artifacts exist. Usage: `python scripts/e2e_run.py --runs 20`.

- `docs/ppt/` : the presentation (problem, architecture, model strategy, sovereign proof, demo, roadmap).
- `demo/talk_track.md` : what to say during each scenario, minute by minute.
- Backup video of a perfect run of all 3 scenarios.

Dev B checklist

- ☐ B0 Mock server + fixtures
- ☐ B1 Contract tests
- ☐ B2 API client
- ☐ B3 UI shell
- ☐ B4 Timeline
- ☐ B5 Badge, plan, artifacts
- ☐ B6 Demo data (to Dev A by Day 1)
- ☐ B7 Network + audit pages
- ☐ B8 Resilience
- ☐ B9 Demo pack

Sync points between Dev A and Dev B

When	What happens
End of Day 0	Contract frozen, both environments pass Step 1 and Step 2
B6 done	Dev A gets sample files for A4 and A6
A3 done	Early integration: Dev B sets <code>WB_API_URL</code> to port 8000 for 30 minutes, runs contract tests, then goes back to the mock
A8 done	Full integration (Step 8)

Step 7: Model strategy (PPT vs live demo)

The PPT shows the production lineup for an MRPL GPU server; the live demo runs small models on a CPU laptop. The code is the same; only `config/models.yaml` changes.

7.1 In the PPT: production server (for example 4x H100 80GB)

Role	Model	Why
Main brain: planning, reasoning, writing	gpt-oss-120b on vLLM	117B parameters, 5.1B active, fits one 80GB GPU, Apache 2.0 license
Vision and documents	Qwen3.6-27B	Native image input, 262K token context
Coding	Qwen3-Coder-Next	Large self-hosted coder, needs about 64GB+
Search	bge-m3 + bge-reranker-v2-m3	Multilingual retrieval
Future slot	DeepSeek-V4-Flash	MIT license; shows adding a model is a YAML edit
Serving	vLLM	Many users at once

Before the final PPT, open each model card and re-check the license and size. This field changes every month.

7.2 In the live demo: our 16 GB CPU laptops

Role	Ollama model	Size
General + vision + agent	<code>qwen3.5:4b</code>	about 3 to 4 GB
Coder	<code>qwen2.5-coder:3b</code>	about 2 GB
Embeddings	<code>bge-m3</code>	about 1.2 GB

Why these: qwen3.5 does text, images and tool calling in one model, which saves RAM. A separate coder model means the router visibly picks different models for different tasks. Stable and fast beats powerful on stage.

Fallback if the speed test was below 4 tokens per second: use `qwen3.5:2b` and `qwen2.5-coder:1.5b`.

7.3 CPU speed settings

Setting	Value	Why
<code>WB_THINK</code>	false	Thinking mode adds a minute or more per answer on CPU
<code>WB_NUM_CTX</code>	8192	Bigger context uses more RAM and is slower
<code>WB_TEMPERATURE</code>	0.2	More consistent answers for the demo
Image size	max 1024 px	Vision on CPU takes about 20 to 60 seconds per image
OCR first	Tesseract before vision	Tesseract takes 1 to 2 seconds per page

7.4 The PPT message

Our live demo runs on an ordinary CPU laptop with no GPU and no internet. On an MRPL GPU server, the same code runs gpt-oss-120b and Qwen3.6 by changing one config file.

This turns our hardware limit into proof that the system is hardware-independent.

Step 8: Integration, hardening and demo day

Integration starts when A8 is done: contract tests must pass 100% on the real backend, then we run 20 clean runs in a row before the demo.

8.1 Integration handshake

1. Both: merge `main` into your branch. Do not fix anything in the other dev's folders.
2. Dev A: start the real backend on port 8000. Run `python scripts/doctor.py`, all PASS.
3. Dev B: run `pytest tests/contract --base-url http://127.0.0.1:8000`. It must be 100% green. Any failure is a Dev A bug to fix first.
4. Dev B: set `WB_API_URL=http://127.0.0.1:8000` in `.env` and restart the UI.
5. Together: run Scenario B (fastest), then C, then A. Compare outputs with `demo/expected.md`.

6. Merge both branches into `main`, then tag: `git tag v1.0-integrated` and `git push --tags`.
7. On the second laptop: fresh clone, install from the lock, run all 3 scenarios. This proves the setup is repeatable.

8.2 One-click start: `scripts/start_demo.ps1`

The script does these steps in this exact order:

1. Check Ollama answers at `127.0.0.1:11434`. If not, start it and wait.
2. Check Docker is running and `wb-sandbox:1.0` exists.
3. Start the backend in a new window. Wait until `/api/health` says `ok`.
4. Call `/api/admin/prewarm`.
5. Start the UI and open `http://127.0.0.1:8501` in the browser.
6. Print READY in green.

8.3 Crash-proofing rules

1. Every model call has a timeout. Every tool has try/except that creates an `error` event.
2. The worker thread catches all exceptions. A failed task becomes status `failed` with `ErrorInfo`; the server keeps running.
3. AI JSON is validated with Pydantic and retried once with the error. If it still fails, the guided flow takes over that step.
4. Upload size and file type are checked.
5. The UI never calls the backend without a timeout, and shows a banner instead of crashing.
6. `logs/backend.log` uses a rotating file handler so it never fills the disk.

8.4 Hardening tests (all must pass)

Test	How	Pass when
Soak	<code>python scripts/e2e_run.py --runs 20</code>	20 of 20 runs succeed
Offline	Wi-Fi off + <code>firewall_block.ps1</code> , run all 3 scenarios	All pass, monitor shows 0 external

Test	How	Pass when
Kill Ollama	Quit Ollama during Scenario A	Task shows <code>MODEL_UNAVAILABLE</code> , UI banner, next run works after restart
Kill backend	Close the backend window	UI banner, recovers after <code>start_demo.ps1</code>
Cold boot	Restart laptop, run only <code>start_demo.ps1</code>	READY in under 3 minutes
Fresh machine	Clone on the second laptop	Same results

8.5 Demo day checklist

- ☐ Both laptops charged, chargers packed; laptop 2 fully set up as live backup
- ☐ Close every other app (RAM)
- ☐ Run `firewall_block.ps1` as admin, turn Wi-Fi off
- ☐ Run `start_demo.ps1` 15 minutes before; one warm-up run of Scenario B
- ☐ Guided mode toggle visible; if the free agent goes off track, switch to guided mode
- ☐ Show the network panel and click "Try to reach Google" (it must fail)
- ☐ Backup video on USB stick and on desktop
- ☐ After the demo: run `firewall_restore.ps1`

8.6 Demo order (suggested)

1. Show the network panel: 0 external connections, probe fails.
2. Scenario B (coding, 1 to 2 minutes): show the router picking the coder model and tests passing.
3. Scenario C (P&ID, 1 to 3 minutes): show the vision path and the Excel output.
4. Scenario A (approval note, 2 to 5 minutes): talk through the agent steps while it runs, then open the Word file.
5. End on the network panel again: still 0, and the audit log shows only 127.0.0.1 calls.

Troubleshooting and quick commands

Most problems on our laptops come from RAM, old versions, or telemetry. Check this table before asking for help, and paste the exact error in the team chat if it is not here.

Common problems

Problem	Why it happens	Fix
Laptop freezes or model crashes	Not enough RAM	Close Chrome and other apps; switch to <code>qwen3.5:2b</code>
Very slow answers	Thinking mode on or big context	Check <code>WB_THINK=false</code> , lower <code>WB_NUM_CTX</code>
Tool call printed as text	Known small-model issue	A1 fallback parser; do not update Ollama before the demo
Broken JSON from model	Small models make mistakes	Use <code>chat_json</code> with schema; one retry with the error
Network monitor shows external call	A library sends telemetry	Check telemetry lines in <code>.env</code> ; Chroma <code>anonymized_telemetry=False</code>
OCR text is garbage	Low scan quality	Render at 200 to 300 DPI, fall back to vision model
P&ID tags missed	Symbols too small	4-tile split in A4
Docker not running	Docker Desktop closed or WSL stopped	Open Docker Desktop, wait for Engine running
Docker uses too much RAM	WSL limit missing	Check <code>.wslconfig</code> , run <code>wsl --shutdown</code>
<code>python</code> opens Microsoft Store	App alias	Turn off python aliases in Settings
Scripts disabled in PowerShell	Execution policy	<code>Set-ExecutionPolicy -Scope CurrentUser RemoteSigned</code>
Import error <code>shared.contracts</code>	Running from wrong folder	Run commands from the repo root with venv active

Problem	Why it happens	Fix
UI shows version mismatch	Old contract on one side	Pull <code>main</code> on both laptops, restart backend and UI
Merge conflict	Someone edited the other track's folder	Keep the owner's version; follow Step 3 ownership

Quick commands

What	Command
Activate venv	<code>.\.venv\Scripts\Activate.ps1</code>
Health check	<code>python scripts/doctor.py</code>
Real backend	<code>uvicorn backend.main:app --host 127.0.0.1 - -port 8000</code>
Mock server	<code>uvicorn mock.mock_server:app --host 127.0.0.1 --port 8001</code>
UI	<code>streamlit run ui/app.py --server.address 127.0.0.1 --server.port 8501</code>
All tests	<code>pytest -q</code>
Contract tests (mock)	<code>pytest tests/contract --base-url http://127.0.0.1:8001</code>
Contract tests (real)	<code>pytest tests/contract --base-url http://127.0.0.1:8000</code>
Ingest SOPs	<code>python scripts/ingest.py</code>
Soak test	<code>python scripts/e2e_run.py --runs 20</code>
Loaded models	<code>ollama ps</code>
Block internet (admin)	<code>.\scripts\firewall_block.ps1</code>
Restore internet (admin)	<code>.\scripts\firewall_restore.ps1</code>
Start full demo	<code>.\scripts\start_demo.ps1</code>